



UNIVERSIDADE FEDERAL DO CEARÁ
CENTRO DE XXXXXXXXX
DEPARTAMENTO DE XXXXXXXXXX
CURSO DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

BILLY GRAHAN ALVES RODRIGUES

ALGORITMOS DE BUSCA PARA IDENTIFICAÇÃO ROTAS ENTRE ATIVOS NA
ZELADORIA PÚBLICA NO CONTEXTO DE CIDADES INTELIGENTES

RUSSAS

2025

BILLY GRAHAN ALVES RODRIGUES

ALGORITMOS DE BUSCA PARA IDENTIFICAÇÃO ROTAS ENTRE ATIVOS NA
ZELADORIA PÚBLICA NO CONTEXTO DE CIDADES INTELIGENTES

Trabalho de Conclusão de Curso apresentado ao
Curso de Graduação em Ciência da Computação
do Centro de XXXXXXXX da Universidade
Federal do Ceará, como requisito parcial à
obtenção do grau de bacharel em Ciência da
Computação.

Orientador: Prof. Dr. Mayrton Dias de
Queiroz

RUSSAS

2025

BILLY GRAHAN ALVES RODRIGUES

ALGORITMOS DE BUSCA PARA IDENTIFICAÇÃO ROTAS ENTRE ATIVOS NA
ZELADORIA PÚBLICA NO CONTEXTO DE CIDADES INTELIGENTES

Trabalho de Conclusão de Curso apresentado ao
Curso de Graduação em Ciência da Computação
do Centro de XXXXXXXX da Universidade
Federal do Ceará, como requisito parcial à
obtenção do grau de bacharel em Ciência da
Computação.

Aprovada em:

BANCA EXAMINADORA

Prof. Dr. Mayrton Dias de Queiroz (Orientador)
Universidade Federal do Ceará (UFC)

Prof. Dr. XXXXXXXX XXXXXXXX XXXXXXXX
Universidade do Membro da Banca Dois (SIGLA)

Prof. Dr. XXXXXXXX XXXXXXXX XXXXXXXX
Universidade do Membro da Banca Três (SIGLA)

Prof. Dr. XXXXXXXX XXXXXXXX XXXXXXXX
Universidade do Membro da Banca Quatro (SIGLA)

À minha família, por sua capacidade de acreditar em mim e investir em mim. Mãe, seu cuidado e dedicação foi que deram, em alguns momentos, a esperança para seguir. Pai, sua presença significou segurança e certeza de que não estou sozinho nessa caminhada.

“O sonho é que leva a gente para frente. Se a gente for seguir a razão, fica aquietado, acomodado.”

(Ariano Suassuna)

RESUMO

A gestão eficiente de ativos públicos municipais constitui um desafio logístico para as administrações públicas, especialmente quando demanda visitas periódicas para inspeção e manutenção de equipamentos urbanos distribuídos por diferentes regiões da cidade. Diente desta problemática, o objetivo geral desse trabalho consiste em identificar uma alternativa capaz de encontrar rotas eficientes entre os ativos públicos aplicando o Problema do Caixeiro Viajante ao contexto da gestão pública municipal. Foi realizada uma revisão sistemática da literatura para identificar as principais abordagens metodológicas e algoritmos utilizados em problemas de otimização de rotas em ambientes urbanos. A metodologia envolveu a caracterização do problema, a identificação de técnicas heurísticas adequadas e a análise comparativa de algoritmos utilizando dados de ativos municipais. Os resultados demonstram a viabilidade da aplicação de técnicas de otimização para redução de custos operacionais, economia de tempo de deslocamento e melhoria na eficiência dos serviços prestados à população. A solução proposta insere-se no contexto das cidades inteligentes, contribuindo para a modernização da gestão pública em municípios de pequeno e médio porte, onde as limitações orçamentárias tornam essencial o uso eficiente dos recursos disponíveis. Este trabalho evidencia que a otimização de rotas constitui ferramenta estratégica para maximizar a eficiência operacional da administração pública, promovendo maior agilidade no atendimento às demandas de zeladoria urbana e fortalecendo a qualidade dos serviços municipais.

Palavras-chave: Otimização de rotas. Problema do Caixeiro Viajante. Gestão de ativos públicos. Heurísticas. Cidades inteligentes.

ABSTRACT

Efficient management of municipal public assets represents a logistical challenge for public administrations, especially when it requires periodic visits for the inspection and maintenance of urban equipment distributed across different areas of the city. This work proposes the development of an algorithmic solution based on heuristic techniques for optimizing visitation routes to public assets, applying the Traveling Salesman Problem to the context of municipal public management. A systematic literature review was conducted to identify the main methodological approaches and algorithms used in route optimization problems in urban environments and smart cities. The methodology involved characterizing the problem, identifying suitable heuristic techniques, and performing a comparative analysis of algorithms using municipal asset data. The results demonstrate the feasibility of applying optimization techniques to reduce operational costs, save travel time, and improve the efficiency of services provided to the population. The proposed solution aligns with the smart city framework, contributing to the modernization of public management in small and medium-sized municipalities, where budget constraints make the efficient use of available resources essential. This work highlights that route optimization is a strategic tool for maximizing the operational efficiency of public administration, promoting greater agility in addressing urban maintenance demands and strengthening the quality of municipal services.

Keywords: Route optimization. Traveling Salesman Problem. Public asset management. Heuristics. Smart cities.

LISTA DE FIGURAS

Figura 1 – Representação de um grafo não dirigido. (a) Um grafo não dirigido G com cinco vértices e sete arestas. (b) Uma representação de G por lista de adjacências. (c) A representação de G por matriz de adjacências.	24
Figura 2 – Exemplo de execução da Busca em Largura (BFS) em um grafo	25
Figura 3 – Exemplo de execução da Busca em Profundidade (DFS) em um grafo	26
Figura 4 – Exemplo de aplicação do algoritmo A^* em um grafo com arestas ponderadas.	28
Figura 5 – Resultados da Questões de Pesquisa (QP)1.	32
Figura 6 – Resultados da QP2.	33
Figura 7 – Passos realizados durante o desenvolvimento desse trabalho.	35
Figura 8 – Quantidade de artigos obtidos após a pesquisa nas Bases de Dados, o Download e aplicação dos Critérios de Inclusão e Exclusão.	36
Figura 9 – Representação em grafo da malha viária de um bairro real, com destaque para os pontos de interseção (vértices) e trechos de ruas (arestas).	38

LISTA DE TABELAS

Tabela 1 – Quantidade de artigos em cada base de dados.	31
Tabela 2 – Resultados preliminares dos algoritmos de busca em diferentes instâncias do problema	49

LISTA DE QUADROS

Quadro 1 – Bases de dados selecionadas.	30
Quadro 2 – Cronograma de execução - TCC	46

LISTA DE ALGORITMOS

Algoritmo 1 – Melhor Rota	39
Algoritmo 2 – Roleta de Seleção	40
Algoritmo 3 – Gerar Solução	41

LISTA DE ABREVIATURAS E SIGLAS

<i>A*</i>	A estrela
<i>API</i>	<i>Application Programming Interface</i>
<i>BFS</i>	<i>Breadth-First Search</i>
<i>CIM</i>	<i>City Information Modeling</i>
<i>DAG</i>	<i>Directed Acyclic Graph</i>
<i>DFS</i>	<i>Depth-First Search</i>
<i>FIFO</i>	<i>First In, First Out</i>
<i>GRASP</i>	<i>Greedy Randomized Adaptive Search Procedure</i>
<i>ITS</i>	<i>Intelligent Transportation Systems</i>
<i>IoT</i>	<i>Internet of Things</i>
<i>LIFO</i>	<i>Last In, First Out</i>
<i>ORS</i>	<i>OpenRouteService</i>
<i>TSP</i>	<i>Traveling Salesman Problem</i>
<i>UCS</i>	<i>Uniform-Cost Search</i>
QP	Questões de Pesquisa

LISTA DE SÍMBOLOS

b	Fator de ramificação
C^*	Custo da solução ótima
E	Conjunto de arestas de um grafo
e	Aresta de um grafo
$f(n)$	Função de avaliação no algoritmo A* ($f(n) = g(n) + h(n)$)
G	Grafo, definido como $G = (V, E)$
$g(n)$	Custo do caminho da origem até o vértice n
$h(n)$	Função heurística que estima o custo de n até o objetivo
n	Vértice ou nó em um grafo
O	Notação Big-O para complexidade de tempo ou espaço
u	Vértice em um grafo
v	Vértice em um grafo
V	Conjunto de vértices de um grafo
w	Função peso que atribui valores às arestas ($w : E \rightarrow \mathbb{R}$)
ε	Menor custo de aresta

SUMÁRIO

1	INTRODUÇÃO	15
1.1	Objetivos	15
1.1.1	<i>Objetivos específicos</i>	15
1.2	Justificativa	16
1.3	Organização do Trabalho	17
2	FUNDAMENTAÇÃO TEÓRICA	18
2.1	Cidades inteligentes	18
2.1.1	<i>Eixos das Cidades Inteligentes</i>	18
2.1.1.1	<i>Governança Inteligente</i>	18
2.1.1.2	<i>Mobilidade Urbana Inteligente</i>	19
2.1.1.3	<i>Urbanismo e Planejamento Urbano</i>	19
2.2	Gestão de ativos públicos	20
2.3	Zeladoria Urbana	20
2.4	Métodos de Otimização	21
2.4.1	<i>Métodos Exatos</i>	21
2.4.2	<i>Heurísticas</i>	22
2.4.3	<i>Meta-heurísticas</i>	23
2.5	Teoria dos Grafos	23
2.6	Algoritmos de Busca	24
2.6.1	<i>Busca em Largura (BFS)</i>	24
2.6.2	<i>Busca em Profundidade (DFS)</i>	25
2.6.3	<i>Busca de Custo Uniforme</i>	25
2.6.4	<i>Algoritmo A* (A Estrela)</i>	27
3	REVISÃO DA LITERATURA	29
3.1	Contextualização	29
3.2	Planejamento	29
3.2.1	<i>Definição de Questões de Pesquisa (QP)</i>	29
3.2.2	<i>Definição da string de busca</i>	30
3.2.3	<i>Definições de bases de dados</i>	30
3.2.4	<i>Definição dos critérios de inclusão e exclusão</i>	30

3.3	Condução	31
3.4	Resultados	31
3.5	Considerações Finais	33
4	METODOLOGIA	35
4.1	Contextualização	35
4.2	Revisão da Literatura	36
4.3	Obtenção das Instâncias	37
4.4	Definição dos Algoritmos	38
4.4.1	<i>Estrutura Geral do Algoritmo</i>	38
4.4.2	<i>Mecanismo de Seleção Probabilística</i>	39
4.4.3	<i>Algoritmos de Busca em Grafos</i>	40
4.4.4	<i>Integração com APIs de Roteamento</i>	42
4.5	Implementação dos Algoritmos	42
4.5.1	<i>Representação do Grafo</i>	43
4.5.2	<i>Padronização dos Experimentos</i>	43
4.6	Análise comparativa dos experimentos	43
4.6.1	<i>Métricas de Avaliação</i>	44
4.6.2	<i>Estratégia de Comparação</i>	44
4.6.3	<i>Análise de Viabilidade</i>	44
4.6.4	<i>Procedimento Experimental</i>	45
4.7	Considerações Finais	45
5	RESULTADOS	47
5.1	Ambiente Computacional	47
5.2	Execução dos Algoritmos	47
5.2.1	<i>Configuração dos Experimentos</i>	48
5.2.2	<i>Resultados Preliminares</i>	48
	REFERÊNCIAS	50

1 INTRODUÇÃO

A gestão eficiente dos ativos públicos constitui um desafio constante para as administrações municipais, especialmente quando demanda visitas periódicas para inspeção, manutenção ou levantamento de informações. Esses ativos — como praças, escolas, unidades de saúde, postes de iluminação, equipamentos urbanos, entre outros — encontram-se com algum Problema por diferentes regiões da cidade, exigindo um planejamento logístico criterioso que minimize o tempo de deslocamento e os custos operacionais das equipes responsáveis.

No contexto das cidades inteligentes, a otimização de rotas torna-se fundamental para a eficiência operacional dos serviços públicos. A ausência de planejamento adequado resulta em desperdício de recursos, aumento do tempo de resposta às demandas da população e ineficiência na prestação de serviços essenciais. Este cenário é particularmente crítico em municípios de pequeno e médio porte, onde as limitações orçamentárias e de infraestrutura são mais acentuadas.

O desafio consiste em determinar a rota mais eficiente para que profissionais da prefeitura percorram todos os pontos de visitação necessários e retornem ao ponto de origem, minimizando a distância total percorrida e, conseqüentemente, o tempo e os custos operacionais.

1.1 Objetivos

Diante do contexto apresentado, o objetivo geral deste trabalho consiste em desenvolver e avaliar um algoritmo capaz de gerar rotas otimizadas para a visitação de ativos públicos por profissionais da administração municipal, aplicando técnicas heurísticas para resolver o Problema do Caixeiro Viajante *Traveling Salesman Problem (TSP)* adaptado à realidade da gestão pública.

1.1.1 Objetivos específicos

Para alcançar o objetivo geral, estabelecem-se os seguintes objetivos específicos:

- Realizar uma revisão sistemática da literatura para identificar os principais trabalhos relacionados à otimização de rotas em contextos urbanos e cidades inteligentes, bem como as técnicas e algoritmos empregados;
- Identificar o Problema do Caixeiro Viajante e suas variantes, contextualizando suas aplicações na gestão de ativos públicos municipais;
- Obter um algoritmo heurístico para geração de rotas otimizadas, considerando as especifi-

cidades da gestão pública;

- Obter uma avaliação do desempenho do algoritmo desenvolvido por meio de análise comparativa utilizando dados de ativos municipais;

1.2 Justificativa

A relevância deste trabalho fundamenta-se em múltiplas dimensões que abrangem aspectos técnicos, econômicos e sociais. Do ponto de vista técnico, o Problema do Caixeiro Viajante representa um desafio clássico da ciência da computação, classificado como NP-difícil, cuja solução exata torna-se impraticável para instâncias de grande porte. A aplicação de técnicas heurísticas permite obter soluções de boa qualidade em tempo computacional razoável, viabilizando sua implementação em contextos práticos.

No âmbito econômico, a otimização de rotas contribui diretamente para a redução de custos operacionais no setor público. A economia gerada pela minimização das distâncias percorridas reflete-se na diminuição do consumo de combustível, na redução do desgaste dos veículos e no aproveitamento mais eficiente do tempo dos profissionais. Essa otimização representa uma ferramenta estratégica para maximizar a eficiência dos recursos disponíveis.

Sob a perspectiva social, a melhoria na eficiência operacional da administração pública impacta diretamente a qualidade dos serviços prestados à população. Rotas otimizadas permitem maior agilidade no atendimento às demandas de manutenção urbana, reduzindo o tempo de resposta e aumentando a capacidade de atendimento das equipes municipais. Esse ganho de eficiência fortalece a confiança da população nas instituições públicas e contribui para a melhoria da qualidade de vida nos centros urbanos.

Além disso, este trabalho insere-se no contexto contemporâneo das cidades inteligentes, onde a aplicação de tecnologias da informação e técnicas de otimização tornam-se instrumentos essenciais para a modernização da gestão pública. A solução proposta pode ser adaptada e replicada em diversos municípios, especialmente aqueles de pequeno e médio porte, democratizando o acesso a ferramentas tecnológicas que promovam eficiência administrativa e transparência na gestão dos recursos públicos.

1.3 Organização do Trabalho

Este trabalho está estruturado em cinco capítulos, organizados de forma a proporcionar uma compreensão progressiva e sistemática do tema abordado.

O Capítulo 1 apresenta a contextualização do problema, os objetivos da pesquisa, a justificativa e a organização geral do trabalho, fornecendo uma visão panorâmica dos elementos centrais desta investigação.

O Capítulo 2 expõe a fundamentação teórica necessária para a compreensão do trabalho, abordando os conceitos de cidades inteligentes, gestão de ativos públicos, zeladoria urbana, métodos de otimização e teoria dos grafos. São apresentados os principais algoritmos de busca e técnicas heurísticas relevantes para a resolução do problema proposto.

O Capítulo 3 descreve a revisão sistemática da literatura realizada, detalhando as etapas de planejamento, condução e análise dos resultados. São apresentados os principais trabalhos relacionados à otimização de rotas em contextos urbanos e cidades inteligentes, identificando estratégias metodológicas, algoritmos empregados e lacunas de pesquisa.

O Capítulo 4 apresenta a metodologia empregada no desenvolvimento da solução, descrevendo o algoritmo proposto, os dados utilizados, os procedimentos de implementação e os critérios de avaliação de desempenho.

Por fim, o Capítulo 5 consolida as conclusões do trabalho, sintetizando os principais resultados obtidos, discutindo as limitações da pesquisa e apontando direções para trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Nesta seção serão abordados os assuntos e conceitos utilizados atualmente para soluções de melhoria de bem-estar da população, como: cidades inteligentes, gestão de ativos públicos, zeladoria urbana, e os algoritmos de busca.

2.1 Cidades inteligentes

O conceito de cidades inteligentes (do inglês, *smart cities*) é definido como um ambiente urbano que utiliza ferramentas de tecnologia da informação e comunicação (TIC) para melhorar a infraestrutura urbana, a utilização inteligente dos recursos, a mobilidade, a governança e os serviços públicos, tendo como principal beneficiado o cidadão (WEISS *et al.*, 2015). Embora o conceito de cidades inteligentes seja amplamente implementado em grandes centros urbanos e metrópoles, é possível realizar sua adaptação a pequenos e médios municípios brasileiros. O uso de tecnologias em tempo real, automação de processos e a participação ativa da população possibilitam a adaptação e a melhoria da qualidade da gestão pública local (SANTIAGO, 2023).

As cidades inteligentes estruturam-se em múltiplos eixos de atuação que, integrados, promovem o desenvolvimento urbano sustentável e centrado no cidadão. Esses eixos abrangem diferentes dimensões da vida urbana e demandam soluções tecnológicas específicas para enfrentar os desafios contemporâneos das cidades (CARVALHO, 2024).

2.1.1 Eixos das Cidades Inteligentes

Nessa seção, serão abordados os eixos de atuação das cidades inteligentes, sendo eles a governança inteligente, a mobilidade urbana inteligente e o urbanismo e planejamento urbano.

2.1.1.1 Governança Inteligente

A governança inteligente refere-se à aplicação de tecnologias digitais para promover a participação cidadã, transparência administrativa e eficiência na gestão pública. Neste eixo, os sistemas de informação permitem que gestores públicos tomem decisões baseadas em dados, realizem o monitoramento de serviços em tempo real e promovam a interação direta com a

população (WEISS *et al.*, 2015). Ferramentas como *dashboards* interativos e plataformas de dados abertos são exemplos de aplicações que facilitam o acesso à informação e fortalecem a democracia participativa (DIAS *et al.*, 2019). A gestão orientada por dados contribui para a otimização de recursos públicos e para o planejamento estratégico de longo prazo, especialmente em contextos de restrição orçamentária.

2.1.1.2 Mobilidade Urbana Inteligente

A mobilidade urbana inteligente integra sistemas de transporte, tecnologias de geolocalização e análise de dados para otimizar o deslocamento de pessoas e bens nas cidades. Sistemas Inteligentes de Transporte (*Intelligent Transportation Systems (ITS)*) utilizam sensores, câmeras e algoritmos de análise para monitorar o fluxo de tráfego, identificar congestionamentos e propor rotas alternativas (COSTA, 2023). Além disso, soluções de estacionamento inteligente baseadas em sistemas ciber-físicos e agentes inteligentes contribuem para a redução do tempo de busca por vagas e diminuição da poluição urbana (BOTELHO *et al.*, 2019). A aplicação de técnicas de aprendizado de máquina permite a análise de padrões espaço-temporais da mobilidade, auxiliando no planejamento de intervenções estruturantes no sistema viário (MELONIO, 2021).

2.1.1.3 Urbanismo e Planejamento Urbano

O eixo de urbanismo inteligente envolve o uso de geotecnologias, modelagem de informações urbanas e análise espacial para o planejamento e gestão do território urbano. A adoção de metodologias como *City Information Modeling (CIM)* permite a integração de dados geoespaciais, infraestrutura urbana e informações socioeconômicas em modelos tridimensionais que auxiliam gestores no planejamento territorial e na tomada de decisões (SANTIAGO, 2023). Aplicações *mobile* para relato de problemas urbanos por parte da população, integradas a sistemas de geolocalização, possibilitam o mapeamento colaborativo de demandas e a priorização de intervenções no espaço urbano (SANTOS *et al.*, 2019). A simulação urbana integrada a sistemas multi-agentes e Internet das Coisas (*Internet of Things (IoT)*) permite testar cenários de desenvolvimento urbano antes de sua implementação física (CASTRO *et al.*, 2020).

2.2 Gestão de ativos públicos

Os ativos públicos são recursos, direitos e serviços que são acessíveis e disponíveis para todos os membros de uma sociedade, com diversas finalidades, tais como: infraestrutura, serviços essenciais, bens comuns, patrimônio cultural e recursos naturais. A gestão de ativos públicos consiste no conjunto de práticas e processos que visam controlar, inventariar, manter e planejar a utilização desses recursos de forma eficiente e sustentável ao longo de seu ciclo de vida.

No contexto das cidades inteligentes, a gestão de ativos públicos torna-se estratégica para garantir a qualidade dos serviços urbanos e a otimização dos recursos públicos. O gerenciamento eficaz desses ativos demanda sistemas de informação capazes de permitir o acompanhamento em tempo real, com dados precisos de localização, estado de conservação e histórico de ações realizadas. A integração de tecnologias de geolocalização, sensores *IoT* e plataformas de gerenciamento permite que gestores públicos tenham visibilidade completa sobre o patrimônio municipal (SANTIAGO, 2023).

Soluções informatizadas de gestão de ativos reduzem erros operacionais, previnem retrabalhos e promovem melhor uso dos recursos públicos (STORCK *et al.*, 2017). Isso é particularmente importante em contextos onde há limitação orçamentária, como em municípios de pequeno e médio porte. A implementação de sistemas integrados de gestão possibilita a criação de inventários digitais georreferenciados, o planejamento preventivo de manutenções e a geração de indicadores de desempenho para a tomada de decisão baseada em evidências.

Além disso, a gestão de ativos públicos no contexto de cidades inteligentes permite a integração com sistemas de participação cidadã, onde a população pode reportar problemas relacionados à infraestrutura urbana através de aplicativos móveis (SANTOS *et al.*, 2019). Essa integração entre gestão de ativos e participação popular fortalece a governança inteligente e contribui para a melhoria contínua dos serviços públicos, promovendo maior transparência e efetividade nas ações do poder público (OLIVEIRA, 2020).

2.3 Zeladoria Urbana

A zeladoria urbana aborda um conjunto de atividades que visam manter em boas condições de uso os ambientes públicos para a população, realizando serviços essenciais de manutenção e conservação, tais como: manutenção de ruas, poda de árvores, limpeza de vias,

conservação de praças e calçadas. Em cidades de pequeno porte, especialmente no interior dos estados do Nordeste brasileiro, é possível encontrar desafios como: limitações operacionais, de infraestrutura e recursos humanos especializados (SANTOS *et al.*, 2019).

Os processos nesses contextos costumam não ser automatizados e sistematizados, o que compromete seriamente o planejamento, a priorização das demandas, a transparência das ações dos gestores e pode propiciar o mau uso de verba pública. A implementação de sistemas digitais de gestão de demandas de zeladoria urbana, integrados a tecnologias de geolocalização e priorização inteligente, pode contribuir significativamente para a eficiência operacional e para a melhoria da qualidade dos serviços prestados à população (OLIVEIRA, 2020).

Aplicativos móveis para reporte de problemas urbanos, como buracos em vias, iluminação pública defeituosa e lixo acumulado, têm se mostrado eficazes na mediação entre cidadãos e gestores públicos (SANTOS *et al.*, 2019). Quando integrados a sistemas de gestão de ativos e algoritmos de otimização de rotas, esses aplicativos possibilitam não apenas o registro das demandas, mas também o planejamento eficiente das equipes de manutenção, a priorização baseada em critérios objetivos e o acompanhamento transparente do andamento das solicitações.

2.4 Métodos de Otimização

Problemas de otimização são encontrados em diversas aplicações práticas, por exemplo o planejamento de rotas. Esses problemas buscam encontrar uma solução factível dentre um conjunto de soluções possíveis, de acordo com determinado critério de otimização (CORMEN *et al.*, 2012). Os métodos para resolver problemas de otimização podem ser classificados em diferentes categorias, cada uma com características, vantagens e limitações específicas.

2.4.1 Métodos Exatos

Métodos exatos são aqueles que garantem encontrar a solução ótima para um problema de otimização. Esses métodos exploram sistematicamente o espaço de soluções, avaliando todas as possibilidades ou utilizando técnicas de poda para eliminar soluções que comprovadamente não podem ser ótimas (CORMEN *et al.*, 2012).

Entre as principais técnicas exatas, destacam-se:

- **Enumeração Completa:** Consiste em avaliar todas as soluções possíveis e selecionar a melhor. Embora garanta a solução ótima, torna-se impraticável para problemas com

grande espaço de busca, pois geralmente possuem um crescimento exponencial do número de soluções.

- **Programação Dinâmica:** Técnica que resolve problemas complexos dividindo-os em subproblemas menores e armazenando os resultados desses subproblemas para evitar recálculos (CORMEN *et al.*, 2012). É aplicável quando o problema possui subestrutura ótima (a solução ótima contém soluções ótimas dos subproblemas) e subproblemas sobrepostos.
- **Branch-and-Bound:** Método que particiona o espaço de soluções em subconjuntos (ramificação) e utiliza limitantes para podar ramos que não podem conter a solução ótima. É amplamente utilizado em problemas de otimização combinatória, como o problema do caixeiro viajante.
- **Programação Linear e Inteira:** Técnicas matemáticas que formulam o problema de otimização como um conjunto de equações e inequações lineares. Enquanto a programação linear lida com variáveis contínuas, a programação inteira trata de variáveis discretas, sendo esta última geralmente mais complexa computacionalmente.

A principal limitação dos métodos exatos é a complexidade computacional. Parte dos problemas de otimização pertencem à classe NP-difícil, onde não se conhece algoritmo que encontre a solução ótima em tempo polinomial. Para instâncias de grande porte, o tempo necessário para obter a solução ótima pode ser impraticável, motivando o uso de métodos aproximados (CORMEN *et al.*, 2012).

2.4.2 Heurísticas

Heurísticas são métodos que buscam encontrar boas soluções em tempo computacional razoável, sem necessariamente garantir a otimalidade. Uma heurística é uma regra prática, estratégia ou técnica que simplifica a resolução de problemas complexos (RUSSELL; NORVIG, 2013).

As principais características das heurísticas incluem:

- **Eficiência computacional:** Geralmente possuem complexidade de tempo polinomial, permitindo resolver instâncias grandes em tempo razoável.
- **Qualidade de solução:** Embora não garantam a solução ótima, frequentemente produzem soluções de boa qualidade, próximas ao ótimo.
- **Especificidade:** Parte das heurísticas são desenvolvidas explorando características específicas do problema, tornando-as eficazes para determinados tipos de instâncias.

Exemplos de heurísticas construtivas incluem o algoritmo do vizinho mais próximo para o problema do caixeiro viajante, onde, partindo de um vértice inicial, sempre se escolhe o vértice não visitado mais próximo até completar o circuito. Heurísticas de melhoria, como a busca local, partem de uma solução inicial e iterativamente aplicam modificações locais que melhoram a qualidade da solução (CORMEN *et al.*, 2012).

2.4.3 Meta-heurísticas

Meta-heurísticas são estratégias de alto nível que guiam heurísticas subordinadas na busca por soluções de qualidade. Diferentemente das heurísticas clássicas, as meta-heurísticas são mais genéricas e podem ser aplicadas a diversos problemas de otimização com pequenas adaptações (RUSSELL; NORVIG, 2013).

A escolha entre métodos exatos, heurísticas ou meta-heurísticas depende do contexto da aplicação, considerando fatores como o tamanho da instância, requisitos de qualidade da solução, tempo disponível para computação e recursos computacionais (CORMEN *et al.*, 2012).

2.5 Teoria dos Grafos

Um grafo é uma estrutura matemática utilizada para modelar relações entre objetos, sendo amplamente empregado na ciência da computação para representar redes, mapas e sistemas diversos (CORMEN *et al.*, 2012). Formalmente, um grafo G é definido pelo par ordenado $G = (V, E)$, onde V é um conjunto finito de vértices (ou nós) que representam entidades, e E é um conjunto de arestas que representam as conexões entre os vértices, denotadas como pares (u, v) , onde $u, v \in V$.

Os grafos podem ser classificados quanto à direcionalidade das arestas. Em grafos **não-direcionados**, as arestas estabelecem conexões bidirecionais, onde (u, v) é equivalente a (v, u) , representando relações simétricas como estradas de mão dupla. Já nos grafos **direcionados** (ou dígrafos), as arestas possuem orientação, indicando uma conexão de u para v sem necessariamente existir de v para u , úteis para modelar ruas de mão única ou dependências entre tarefas.

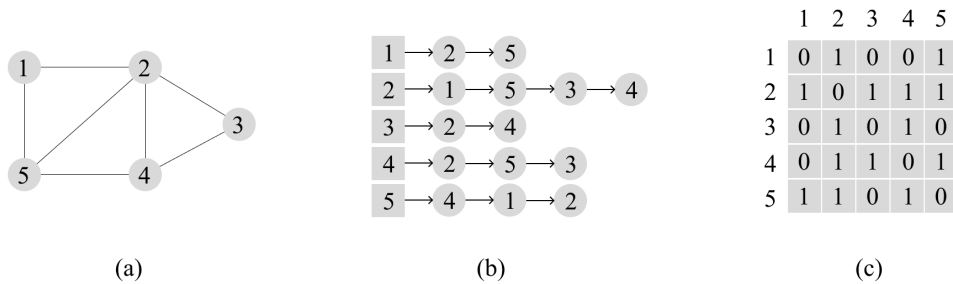
Quanto à ponderação, grafos podem ter arestas com pesos ou custos associados, definidos por uma função $w : E \rightarrow \mathbb{R}$ que atribui valores numéricos representando distâncias, tempos, custos ou capacidades (CORMEN *et al.*, 2012). Conceitos fundamentais incluem

adjacência (vértices conectados por aresta), **grau** (número de arestas incidentes a um vértice), **caminho** (sequência de vértices conectados) e **ciclo** (caminho que retorna ao vértice inicial).

Computacionalmente, grafos podem ser representados de duas formas principais: (i) **Matriz de Adjacência** – matriz A de dimensão $|V| \times |V|$ onde $A[i][j]$ indica a existência e peso de arestas, ocupando espaço $O(V^2)$; (ii) **Lista de Adjacência** – para cada vértice mantém-se uma lista dos adjacentes, mais eficiente para grafos esparsos com complexidade espacial $O(V + E)$ (CORMEN *et al.*, 2012).

A Figura 1 ilustra um exemplo de grafo e sua representação por lista e matriz de adjacências.

Figura 1 – Representação de um grafo não dirigido. (a) Um grafo não dirigido G com cinco vértices e sete arestas. (b) Uma representação de G por lista de adjacências. (c) A representação de G por matriz de adjacências.



Fonte: Elaborado pelo autor (2025).

2.6 Algoritmos de Busca

Nessa seção, serão apresentados alguns dos principais algoritmos de busca em grafos, sendo eles a Busca em Largura (BFS), a Busca em Profundidade (DFS), a Busca de Custo Uniforme (UCS) e o Algoritmo A* (A Estrela).

2.6.1 Busca em Largura (BFS)

A Busca em Largura *Breadth-First Search* (BFS) explora o grafo sistematicamente visitando todos os vértices a uma mesma distância da origem antes de avançar para níveis mais profundos (CORMEN *et al.*, 2012). O algoritmo utiliza uma estrutura de fila *First In, First Out* (FIFO) para controlar a ordem de visitação dos vértices.

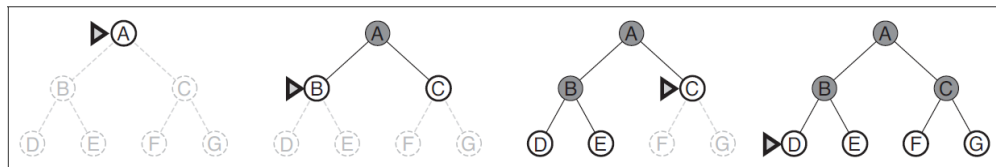
A complexidade de tempo da BFS é $O(V + E)$, onde V é o número de vértices e E é

o número de arestas. O algoritmo garante encontrar o caminho mais curto em termos de número de arestas quando todas têm o mesmo peso. A BFS é particularmente útil para:

- Encontrar o menor caminho em grafos não ponderados;
- Testar a conectividade de um grafo;
- Encontrar todos os vértices alcançáveis a partir de um vértice de origem;
- Detectar ciclos em grafos não direcionados.

A Figura 2 ilustra o funcionamento da busca em largura em um grafo, mostrando a ordem de visita dos vértices nível por nível.

Figura 2 – Exemplo de execução da Busca em Largura (BFS) em um grafo



Fonte: (RUSSELL; NORVIG, 2013).

2.6.2 Busca em Profundidade (DFS)

A Busca em Profundidade *Depth-First Search (DFS)* percorre o grafo explorando o máximo possível cada caminho antes de retroceder (CORMEN *et al.*, 2012). Pode ser implementada recursivamente ou utilizando uma pilha *Last In, First Out (LIFO)*. A DFS também possui complexidade $O(V + E)$.

A DFS é especialmente adequada para:

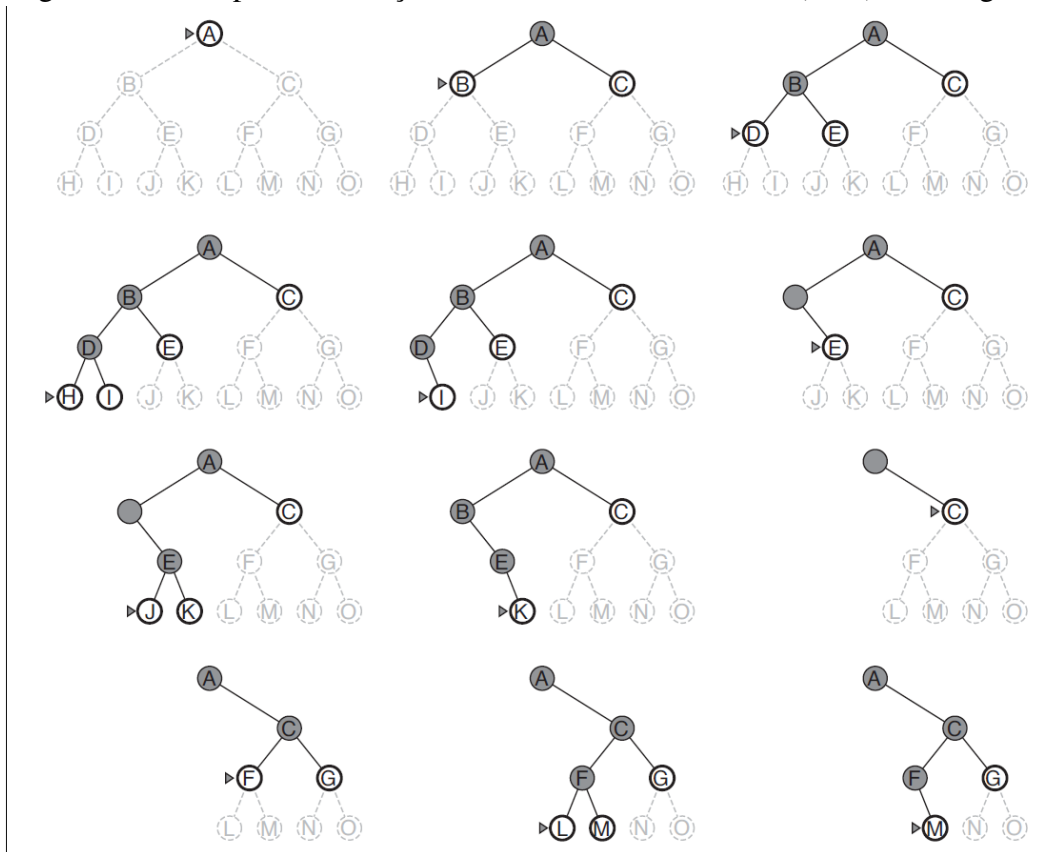
- Detecção de ciclos em grafos direcionados e não direcionados;
- Ordenação topológica em grafos acíclicos direcionados (*Directed Acyclic Graph (DAG)*);
- Decomposição de grafos em componentes fortemente conexos;
- Resolução de quebra-cabeças e problemas que exigem exploração de todas as possibilidades, como labirintos.

A Figura 3 demonstra o processo de busca em profundidade, evidenciando como o algoritmo explora cada ramo até seu limite antes de retroceder.

2.6.3 Busca de Custo Uniforme

A Busca de Custo Uniforme *Uniform-Cost Search (UCS)* é uma estratégia de busca que expande o nó com menor custo de caminho acumulado, sendo uma generalização da BFS

Figura 3 – Exemplo de execução da Busca em Profundidade (DFS) em um grafo



Fonte: (RUSSELL; NORVIG, 2013).

para grafos com arestas de diferentes custos (RUSSELL; NORVIG, 2013). Diferentemente da BFS, que trata todas as arestas igualmente, a UCS considera os pesos associados às arestas para determinar a ordem de expansão dos nós.

O algoritmo utiliza uma fila de prioridade onde os nós são ordenados pelo custo total do caminho da origem $g(n)$, sempre expandindo o nó com menor custo acumulado. A cada expansão, os custos dos nós vizinhos são atualizados se um caminho mais barato for encontrado. A UCS é completa e ótima, garantindo encontrar a solução de menor custo quando todos os custos das arestas são não-negativos (RUSSELL; NORVIG, 2013).

A complexidade de tempo e espaço da busca de custo uniforme é $O(b^{1+\lceil C^*/\epsilon \rceil})$, onde b é o fator de ramificação, C^* é o custo da solução ótima, e ϵ é o menor custo de aresta. Na prática, com implementação eficiente usando *heap* binário, a complexidade é $O((V + E) \log V)$.

A UCS é particularmente adequada para:

- Encontrar caminhos de menor custo em grafos ponderados com pesos não-negativos;
- Problemas de roteamento onde diferentes caminhos têm custos variados;
- Planejamento de rotas considerando distâncias, tempos ou custos operacionais;
- Situações onde não há informação heurística disponível sobre a proximidade do objetivo.

A principal diferença entre a UCS e algoritmos informados como o A^* é que a UCS não utiliza conhecimento adicional (heurística) sobre a distância até o objetivo, expandindo nós apenas com base no custo acumulado desde a origem (RUSSELL; NORVIG, 2013).

2.6.4 Algoritmo A^* (A Estrela)

O algoritmo A estrela (A^*) é uma extensão da busca de custo uniforme que incorpora uma função heurística para guiar a busca em direção ao objetivo (RUSSELL; NORVIG, 2013). A função de avaliação é definida como $f(n) = g(n) + h(n)$, onde $g(n)$ é o custo do caminho da origem até o vértice n , e $h(n)$ é a heurística que estima o custo de n até o objetivo.

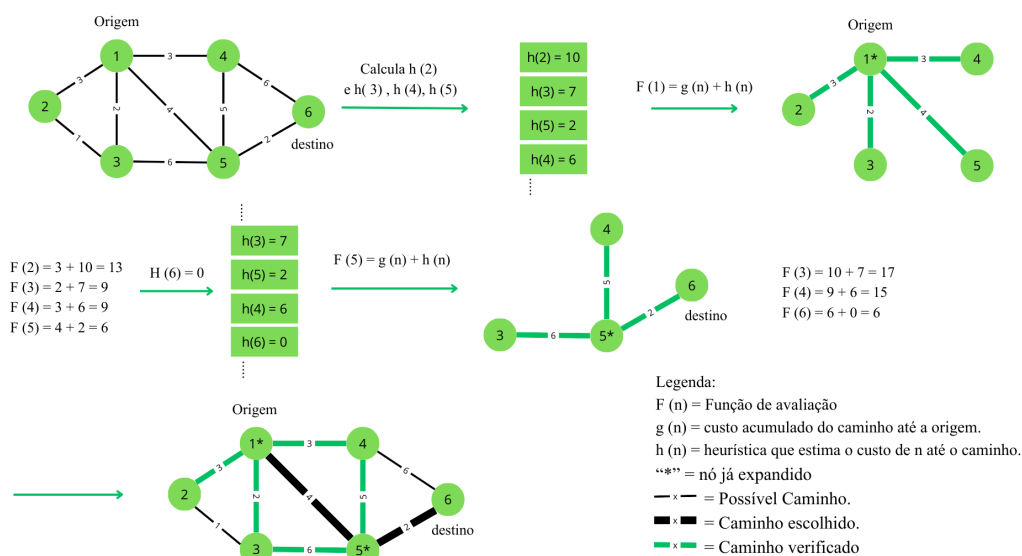
Para que o A^* garanta encontrar o caminho ótimo, a heurística deve ser admissível, ou seja, nunca superestimar o custo real até o objetivo. Heurísticas comuns incluem a distância euclidiana e a distância de Manhattan em espaços bidimensionais (RUSSELL; NORVIG, 2013).

O A^* é particularmente eficiente quando existe uma boa heurística disponível, pois reduz significativamente o número de vértices explorados em comparação com Dijkstra. É amplamente utilizado em:

- Jogos e simulações para movimentação de personagens e entidades;
- Robótica para planejamento de trajetórias;
- Sistemas de navegação com informação geográfica;
- Otimização de rotas considerando múltiplos critérios.

A Figura 4 apresenta um exemplo de aplicação do algoritmo A^* para encontrar o caminho mais curto em um grafo.

Figura 4 – Exemplo de aplicação do algoritmo A* em um grafo com arestas ponderadas.



Fonte: elaborada pelo autor (2025).

3 REVISÃO DA LITERATURA

Neste capítulo serão descritas as etapas realizadas durante a Revisão da Literatura do trabalho.

3.1 Contextualização

Diante do objetivo do trabalho, foi necessário a realização de uma revisão sistemática com o intuito de identificar os principais trabalhos relacionados com essa pesquisa. Esta fase da pesquisa foi organizada em três etapas: Planejamento, Condução e Resultados que serão descritas nas próximas seções.

3.2 Planejamento

Segundo Kitchenham *et al.* (2009), o Planejamento é a etapa de definição de cada passo do protocolo, que consiste em: definição das Questões de Pesquisa; definição da *string* de busca; definição das bases de dados e definição dos critérios de inclusão e exclusão. Sendo assim, nas próximas seções, serão descritos os passos deste protocolo.

3.2.1 Definição de Questões de Pesquisa (QP)

QP1 - Qual a estratégia adotada no trabalho?

- Criação de um *framework*
- Simulação
- Análise de dados
- Abordagem com internet das coisas
- Desenvolvimento/uso de algoritmos de otimização

QP2 - Qual(is) algoritmo(s) adotado(s) no trabalho?

- Algoritmo genético
- Colônia de formigas
- Grasp
- Programação linear
- Algoritmo Híbrido
- Não se aplica

3.2.2 Definição da string de busca

Após a definição das questões de pesquisas, foi definida a seguinte *string* de busca: *"cidades inteligentes"AND "framework"AND "problema do caixeiro viajante"*

3.2.3 Definições de bases de dados

De posse da *string* de busca, foram selecionadas as bases de dados, conforme mostra o Quadro 1:

Quadro 1 – Bases de dados selecionadas.

Base de dados	Link
Google acadêmico	https://www.periodicos.capes.gov.br
Periódicos da capes	https://scholar.google.com.br/
SBC-OpenLib	https://sol.sbc.org.br

Fonte: elaborado pelo autor (2025).

3.2.4 Definição dos critérios de inclusão e exclusão

Para concluir essa etapa do planejamento, foram definidos os critérios de inclusão e exclusão que todos os artigos deverão satisfazer. Os critérios definidos são:

Critérios de inclusão:

- Estudos completos publicados em revistas ou conferências alinhados ao problema de pesquisa;
- Estudos teóricos ou experimentais com o objetivo de apresentar conceitos para o entendimento da área;
- Acessível eletronicamente e ter sido publicado no período de 2015 a 2025.

Critérios de exclusão:

- Estudos que não estejam relacionados ao problema de pesquisa;
- Estudos que não respondem a nenhuma das questões de pesquisa;
- Artigos duplicados, ou seja, aqueles encontrados em mais de uma base de dados;
- Artigos convidados, tutoriais, relatórios técnicos que não passam pelo critério de avaliação das conferências ou revistas;
- Estudos não disponíveis para *download*.

Tabela 1 – Quantidade de artigos em cada base de dados.

Base de dados	Resultados da busca	Download	Inclusão/Exclusão
Google acadêmico	26	13	3
Periódicos da capes	10	6	4
SBC-OpenLib	3	3	1

Fonte: Elaborado pelo autor (2025).

3.3 Condução

Na fase de condução, foi colocado em prática o planejamento definido na seção de Planejamento, dessa forma, a *string* de busca foi inserida em cada base de dados, respeitando as especificações de cada base. De posse dos resultados da busca, foi realizado o *download* dos estudos disponíveis. Em seguida, os trabalhos passaram pelos critérios de inclusão e exclusão. Na Tabela 1, é possível identificar a quantidade de trabalhos em cada base durante a busca, a realização do *download* e a aplicação dos critérios de inclusão e exclusão.

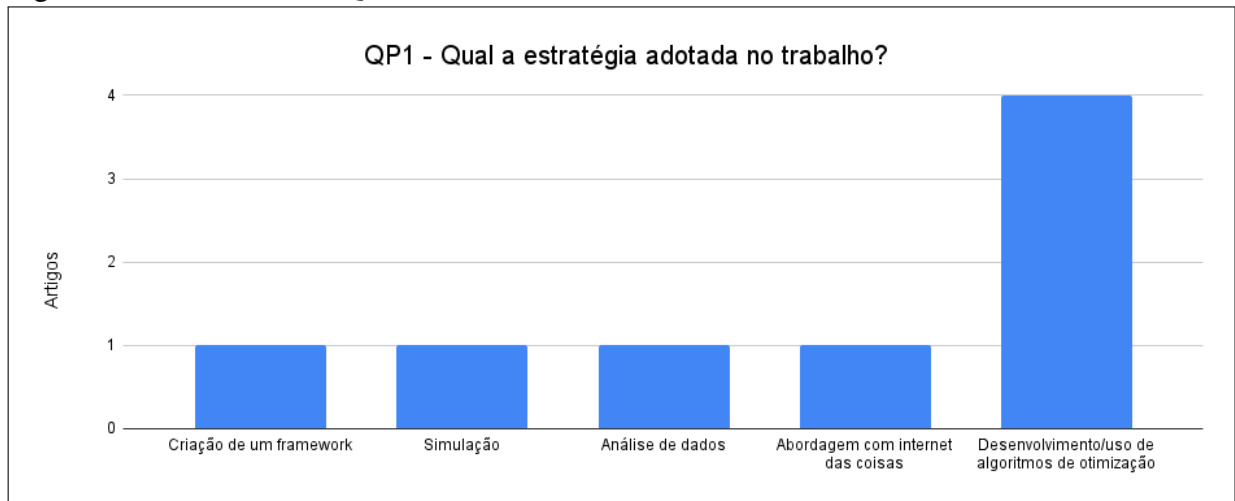
3.4 Resultados

Nesta seção são apresentados e discutidos os resultados obtidos a partir da análise dos trabalhos selecionados, organizados de acordo com as duas questões de pesquisa estabelecidas.

A Figura 5 apresenta um gráfico de barras com os resultados da QP1, referente às estratégias metodológicas adotadas nos trabalhos selecionados. Observa-se que apenas um estudo utilizou a estratégia de criação de *framework*: o trabalho de Silva (2024), no qual os autores desenvolveram uma solução baseada em *Application Programming Interface (API)*s externas para calcular rotas entre pontos estratégicos visando a resolução de Problemas de Roteamento de Veículos.

A estratégia mais prevalente identificada na QP1 foi o desenvolvimento e/ou uso de algoritmos de otimização, totalizando quatro trabalhos. Paz (2023) desenvolveu um algoritmo para otimizar o transporte urbano considerando o menor trajeto no menor tempo possível, evitando gargalos como congestionamentos. Silva *et al.* (2022) utilizou a ferramenta externas de código aberto para resolver o Problema do Caixeiro Viajante em centros urbanos, aplicado ao contexto de entregas dos correios. Silva e Ochi (2017) propôs um algoritmo para resolver uma variação do Problema do Caixeiro Viajante, denominada Caixeiro Alugador, visando otimizar viagens com aluguel de veículos entre cidades. Por fim, Chaves *et al.* (2007) apresentou um método heurístico híbrido para resolver aproximadamente o Problema do Caixeiro Viajante com

Figura 5 – Resultados da QP1.



Fonte: elaborada pelo autor (2025).

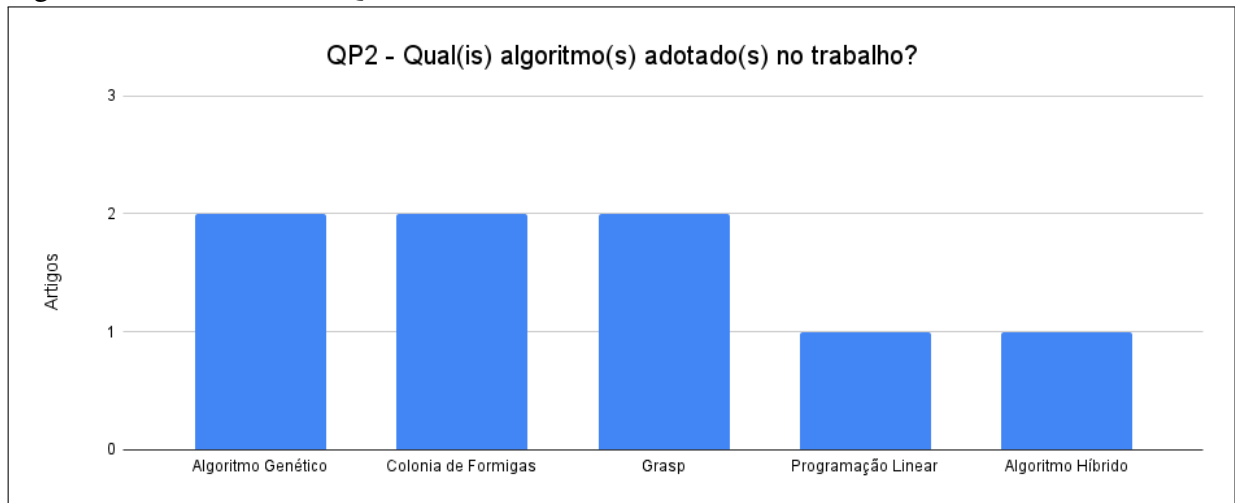
Coleta de Prêmios.

Ainda em relação à QP1, na alternativa de Abordagem com Internet das Coisas, Barth (2016) desenvolveu uma metodologia para elaboração de um sistema computacional baseado em um conjunto de regras definidas por modelagem matemática, visando apoiar o projeto de redes híbridas (ópticas e sem fio) para atendimento das demandas de cidades inteligentes. Na alternativa de simulação, Sati *et al.* (2020) utilizou e comparou três diferentes modelos matemáticos, analisando os benefícios de cada um para a resolução do problema de roteamento de veículos em centros urbanos voltado ao transporte de funcionários. Quanto à alternativa de análise de dados, Georges (2014) apresentou um estudo das rotas de coleta de materiais recicláveis de uma cooperativa, aplicando um modelo conceitual do Problema do Caixeiro Viajante para reordenação dos pontos de coleta.

A Figura 6 apresenta um gráfico de barras com os resultados da QP2, referente aos algoritmos adotados nos trabalhos selecionados. Paz (2023) utilizou o Algoritmo Genético combinado com conceitos de probabilidade para otimização de rotas e, para fins comparativos, também empregou os algoritmos Colônia de Formigas, *Greedy Randomized Adaptive Search Procedure* (GRASP), A* (A estrela) e Dijkstra. Barth (2016) adotou o algoritmo Colônia de Formigas em seu projeto de redes híbridas para cidades inteligentes. Silva e Ochi (2017) e Chaves *et al.* (2007) utilizaram, respectivamente, o Algoritmo Genético e um método híbrido de GRASP com outras técnicas de otimização para refinar a solução construída, ambos aplicados à resolução de variantes do Problema do Caixeiro Viajante. Por sua vez, Sati *et al.* (2020) adotou Programação Linear para resolver o problema de roteamento de veículos em centros urbanos.

Os demais trabalhos, Silva (2024), Silva *et al.* (2022) e Georges (2014), não adotaram

Figura 6 – Resultados da QP2.



Fonte: elaborada pelo autor (2025).

nenhum dos algoritmos especificados na QP2. Estes estudos utilizaram ferramentas externas para resolução dos problemas propostos, tais como OR-Tools (*software* de código aberto do Google), OpenStreetMap, Open Source Routing Machine e/ou modelos conceituais, porém sem implementação direta de algoritmos ou realização de análises comparativas entre diferentes abordagens.

3.5 Considerações Finais

A realização desta revisão sistemática da literatura foi fundamental para mapear o estado da arte em otimização de rotas e gestão de ativos públicos no contexto de cidades inteligentes. Por meio das questões de pesquisa estabelecidas, foi possível identificar as principais estratégias metodológicas adotadas pelos pesquisadores da área, bem como os algoritmos e técnicas de otimização empregados para resolver problemas relacionados ao Problema do Caixeiro Viajante e suas variantes em ambientes urbanos.

Os resultados evidenciaram também a crescente utilização de ferramentas e APIs externas, como OR-Tools, OpenStreetMap e Open Source Routing Machine, que facilitam a implementação de soluções práticas sem a necessidade de reimplementação dos algoritmos.

Esta revisão forneceu subsídios importantes para o desenvolvimento do presente trabalho, orientando tanto a escolha da abordagem metodológica quanto a seleção de técnicas de otimização adequadas ao problema proposto. A análise comparativa dos algoritmos identificados na literatura servirá como referencial para a avaliação da solução desenvolvida, permitindo posicionar a contribuição desta pesquisa no cenário científico atual da área de cidades inteligentes.

e otimização de rotas para gestão pública.

4 METODOLOGIA

Neste capítulo serão descritos os passos realizados durante o desenvolvimento da metodologia adotada neste trabalho, como: a revisão sistemática da literatura; a obtenção das instâncias do problema; os algoritmos desenvolvidos; e os resultados obtidos.

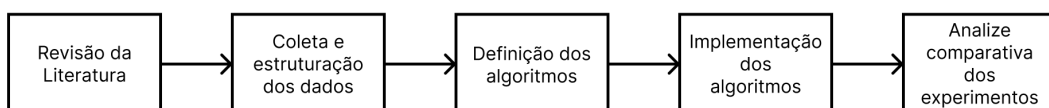
4.1 Contextualização

Este trabalho aborda o problema de otimização de rotas para visitação de ativos públicos municipais no contexto da zeladoria urbana, enquadrando-se como uma aplicação prática do Problema do Caixeiro Viajante (*TSP*). O cenário típico envolve equipes de manutenção que necessitam visitar diversos pontos de interesse distribuídos geograficamente pela cidade, como luminárias públicas, equipamentos urbanos, praças e demais ativos municipais que demandam inspeção ou manutenção periódica.

A relevância do problema justifica-se pela necessidade de otimizar recursos públicos limitados, reduzir custos operacionais com combustível e deslocamento, diminuir o tempo de execução das rotas e aumentar a eficiência dos serviços prestados à população. Em municípios de pequeno e médio porte, onde as restrições orçamentárias são mais severas, a otimização das rotas de zeladoria pode representar ganhos significativos na gestão pública.

A metodologia adotada neste trabalho segue uma abordagem experimental e comparativa, envolvendo as seguintes etapas principais: (i) revisão da literatura; (ii) coleta e estruturação dos dados e instancias do problema; (iii) Definição dos algoritmos; (iv) implementação dos algoritmos; (v) analize comparativa dos experimentos. (vi) Considerações Finais.

Figura 7 – Passos realizados durante o desenvolvimento desse trabalho.



Fonte: elaborada pelo autor (2025).

A escolha por métodos heurísticos justifica-se pela natureza NP-difícil do *TSP*, que torna inviável a obtenção de soluções ótimas em tempo razoável para instâncias de grande porte através de métodos exatos. As heurísticas e meta-heurísticas permitem encontrar soluções de

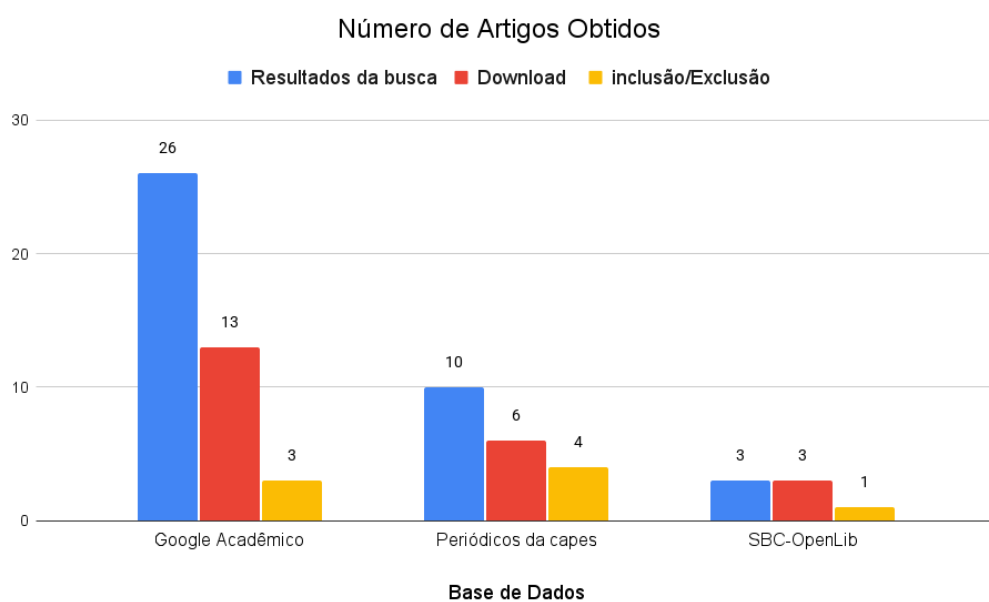
boa qualidade em tempo computacional aceitável, atendendo às necessidades práticas da gestão pública municipal.

4.2 Revisão da Literatura

O primeiro passo realizado no desenvolvimento da metodologia deste trabalho foi a Revisão Sistemática da Literatura, seguindo o protocolo de Kitchenham *et al.* (2009), com o intuito de identificar os principais trabalhos que abordam o problema de otimização de rotas para gestão de ativos públicos e aplicações do *TSP* em contextos urbanos. A descrição detalhada das três etapas realizadas durante a revisão (Planejamento, Condução e Resultados) pode ser observada no Capítulo 3.

Inicialmente, ao realizar a busca dos artigos nas bases de dados (Google Acadêmico, Periódicos da CAPES e SBC-OpenLib) foram encontrados 39 trabalhos. Logo após, foi realizado o *download* desses artigos, em seguida os mesmos passaram pelos critérios de inclusão e exclusão, restando 8 trabalhos primários, conforme mostra a Figura 8. Entre os artigos resultantes, foi possível identificar os métodos de otimização mais utilizados, os principais algoritmos adotados (Algoritmos Genéticos, Colônia de Formigas, *GRASP*, Programação Linear), as estratégias metodológicas empregadas e as métricas de avaliação comumente utilizadas.

Figura 8 – Quantidade de artigos obtidos após a pesquisa nas Bases de Dados, o Download e aplicação dos Critérios de Inclusão e Exclusão.



Fonte: elaborada pelo autor (2025).

A análise dos trabalhos relacionados revelou que a maioria dos estudos adota algoritmos heurísticos e meta-heurísticos para resolução de problemas de roteamento em ambientes urbanos, dada a complexidade computacional de métodos exatos. Identificou-se também o uso crescente de ferramentas externas como OR-Tools, OpenStreetMap e Open Source Routing Machine para implementação prática de soluções. Desta forma, foi possível obter uma visão geral do estado da arte em otimização de rotas urbanas e identificar lacunas que este trabalho busca preencher, particularmente quanto à aplicação específica em zeladoria pública municipal com dados reais de ativos urbanos.

4.3 Obtenção das Instâncias

As instâncias do problema foram obtidas a partir do mapeamento de um bairro da cidade de Russas-CE, posteriormente transformado em uma estrutura de grafo que representa a malha viária urbana. Nesta modelagem, cada ponto de interseção entre ruas constitui um vértice do grafo, enquanto as arestas correspondem aos trechos de ruas que conectam esses pontos de interseção. Esta representação permite capturar de forma fiel a topologia da rede viária, preservando as características geográficas e as restrições de deslocamento reais do ambiente urbano estudado.

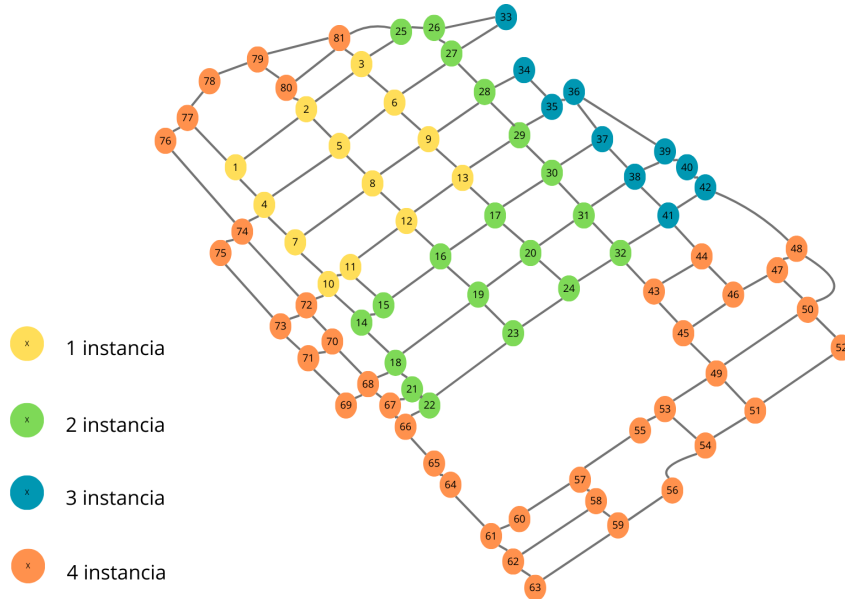
A estrutura de dados escolhida para representar o grafo foi baseada em listas de adjacência, organizadas em duas estruturas complementares. A primeira lista contém as distâncias reais entre os pontos adjacentes, medidas em unidades métricas, permitindo o cálculo preciso dos custos de deslocamento entre os vértices. A segunda lista armazena as coordenadas geográficas (latitude e longitude) de cada ponto, possibilitando a visualização espacial do problema e facilitando a validação das soluções obtidas. Esta abordagem dual proporciona tanto eficiência computacional quanto rastreabilidade geográfica das rotas otimizadas.

Para a composição das instâncias de teste, os ativos públicos a serem visitados são representados por um conjunto de nós selecionados aleatoriamente a partir do grafo completo da malha viária. O objetivo do algoritmo é determinar a rota de menor custo que parte de um nó inicial, visita todos os ativos selecionados exatamente uma vez e retorna ao ponto de partida, caracterizando assim uma instância clássica do *TSP*. Este processo de seleção aleatória permite a geração de múltiplas instâncias de teste com diferentes níveis de complexidade, variando o número e a distribuição espacial dos ativos a serem visitados.

Para a obtenção dos dados geográficos necessários à construção das instâncias, foram

utilizados recursos disponíveis na plataforma *OpenRouteService (ORS)*, que fornece dados de infraestrutura viária baseados no projeto OpenStreetMap. A utilização desta plataforma garante a acurácia dos dados geográficos e das distâncias entre os pontos, elementos essenciais para a validação prática dos resultados obtidos. A Figura 9 ilustra o grafo gerado a partir do mapeamento de um bairro real, evidenciando a malha viária e os pontos de interseção que compõem a estrutura do problema.

Figura 9 – Representação em grafo da malha viária de um bairro real, com destaque para os pontos de interseção (vértices) e trechos de ruas (arestas).



Fonte: elaborada pelo autor (2025).

4.4 Definição dos Algoritmos

A solução proposta neste trabalho baseia-se no desenvolvimento de um algoritmo híbrido que combina conceitos de algoritmos genéticos com técnicas de programação dinâmica para resolver o problema de otimização de rotas de visitação de ativos públicos. O algoritmo completo, detalhado em pseudocódigo, é apresentado a seguir através de três componentes principais da solução: A estratégia de busca pela Melhor Rota (Algoritmo 1), o mecanismo de Roleta de Seleção (Algoritmo 2) e o procedimento de Geração de Solução (Algoritmo 3).

4.4.1 Estrutura Geral do Algoritmo

O algoritmo proposto adota uma abordagem iterativa inspirada em algoritmos genéticos, onde múltiplas soluções são geradas e a melhor delas é selecionada ao longo do processo

evolutivo. A cada iteração, uma nova rota é construída de forma probabilística, permitindo explorar diferentes configurações do espaço de soluções. Este processo de geração repetida de soluções e seleção da melhor alternativa caracteriza a natureza heurística do método, que busca convergir para soluções de alta qualidade sem a necessidade de explorar exaustivamente todo o espaço de busca.

Um elemento fundamental da solução é a aplicação de programação dinâmica para otimizar o cálculo dos caminhos entre os ativos. O algoritmo mantém uma estrutura de memorização (*memoization*) que armazena os caminhos e distâncias já calculados entre pares de ativos, evitando recomputações desnecessárias. Esta estratégia reduz significativamente o custo computacional, especialmente em cenários onde um mesmo par de ativos aparece em múltiplas soluções candidatas ao longo do processo iterativo. A combinação entre a exploração probabilística das rotas e a memorização eficiente dos subcaminhos constitui o diferencial da abordagem proposta.

Algoritmo 1: Melhor Rota

Output: Atualiza a melhor rota encontrada

```

for  $i \leftarrow 0$  to  $iteracoes - 1$  do
     $melhor\_rota\_copia \leftarrow Gerar\_Solucao();$ 
    if  $melhor\_rota = \emptyset$  ou  $melhor\_rota\_copia[-1][1] < melhor\_rota[-1][1]$  then
         $melhor\_rota \leftarrow melhor\_rota\_copia;$ 
    end
end

```

Fonte: Elaborado pelo autor (2025).

4.4.2 Mecanismo de Seleção Probabilística

O componente de Roleta de Seleção (Algoritmo 2) implementa um mecanismo de escolha probabilística para determinar qual ativo será visitado em cada passo da construção da rota. A cada decisão, o algoritmo considera todos os ativos ainda não visitados e calcula um peso inversamente proporcional à distância entre o ponto atual e cada candidato. Quanto menor a distância até um ativo, maior a probabilidade de que ele seja escolhido como próximo destino. Este mecanismo introduz um elemento estocástico na construção das soluções, permitindo que diferentes rotas sejam exploradas ao longo das iterações, ao mesmo tempo em que favorece escolhas localmente eficientes.

Algoritmo 2: Roleta de Seleção

Input: Lista de pares (*objeto*, *distancia*)

Output: Objeto selecionado

foreach (*objeto*, *distancia*) *em roleta* **do**

 if *distancia* = 0 **then**

 | **return** *objeto*

 end

 Adicione *objeto* à lista de objetos;

 Adicione *distancia* à lista de distancias;

end
foreach *distancia em distancias* **do**

 Calcule *peso* = $1/\textit{distancia}$;

 Adicione *peso* à lista de pesos;

end

 Calcule *probabilidades* = $\textit{pesos} / \sum \textit{pesos}$;

Escolha aleatoriamente um objeto com base nas probabilidades;

return objeto escolhido;

Fonte: Elaborado pelo autor (2025).

4.4.3 Algoritmos de Busca em Grafos

Para determinar o caminho ótimo entre dois ativos no grafo da malha viária, o algoritmo 3 utiliza métodos clássicos de busca em grafos. A função genérica *algoritmo(origem, ativo)* pode ser implementada utilizando diferentes estratégias de busca, conforme apresentadas no Capítulo 2. Para os experimentos realizados neste trabalho, foram selecionados os algoritmos *BFS*, *DFS*, *UCS* e *A** para avaliação comparativa de desempenho. No caso do algoritmo *A**, foram utilizadas duas funções heurísticas distintas: a distância euclidiana, calculada a partir das coordenadas cartesianas, e a distância haversiana, que considera a curvatura da Terra para cálculo geodésico entre coordenadas geográficas. Esta diversidade de algoritmos e heurísticas permite avaliar o impacto de cada estratégia na qualidade das rotas geradas e no tempo computacional demandado, identificando a abordagem mais adequada para o contexto de otimização de rotas urbanas em grafos de malha viária.

Algoritmo 3: Gerar Solução

Output: Melhor rota gerada

Inicialize $melhor_rota_copia \leftarrow [(inicio, 0)]$;

$ativos_copia \leftarrow copia(ativos)$;

$origem \leftarrow inicio$;

while $ativos_copia \neq \emptyset$ **do**

Inicialize lista *roleta*;

foreach *ativo em ativos_copia* **do**

if $ativo \notin distancia_entre_ativos[origem]$ **then**

$rota, distancia, nos, ramificacao \leftarrow algoritmo(origem, ativo)$;

Adicione *rota* a *caminho_entre_ativos*;

Atualize $distancia_entre_ativos[origem][ativo] \leftarrow distancia$;

end

Adicione $(ativo, distancia)$ à *roleta*;

end

$elemento \leftarrow Roleta(roleta)$;

Adicione $(elemento, 0)$ à $melhor_rota_copia$;

Remova *elemento* de $ativos_copia$;

$origem \leftarrow elemento$;

end

if $inicio \notin distancia_entre_ativos[origem]$ **then**

$rota, distancia, nos, ramificacao \leftarrow algoritmo(origem, inicio)$;

Adicione *rota* a *caminho_entre_ativos*;

Atualize $distancia_entre_ativos[origem][inicio] \leftarrow distancia$;

end

Adicione $(inicio, 0)$ à $melhor_rota_copia$;

Chame $recalcula_distancias(melhor_rota_copia)$;

return $melhor_rota_copia$;

Fonte: Elaborado pelo autor (2025).

4.4.4 Integração com APIs de Roteamento

Embora o algoritmo proposto opere sobre o grafo explicitamente modelado da malha viária, a arquitetura da solução permite a integração com *APIs* externas de roteamento, como o *ORS*. Esta abordagem alternativa oferece vantagens significativas em contextos práticos, uma vez que elimina a necessidade de mapear e manter atualizada a representação local da rede viária. Serviços como o *ORS* fornecem cálculos de rotas otimizadas considerando não apenas as distâncias geográficas, mas também restrições de trânsito, sentidos de vias, velocidades permitidas e outras características dinâmicas do ambiente urbano.

A utilização de *APIs* externas possibilita que a solução proposta seja facilmente adaptada a diferentes municípios sem a necessidade de construir um novo grafo para cada localidade. Basta fornecer as coordenadas geográficas dos ativos a serem visitados, e o serviço de roteamento retorna os caminhos e distâncias reais entre eles. Esta flexibilidade torna a solução escalável e aplicável em cenários diversos, desde pequenos bairros até municípios inteiros, ampliando significativamente seu potencial de utilização prática na gestão pública municipal.

4.5 Implementação dos Algoritmos

A implementação dos algoritmos propostos foi realizada na linguagem de programação Python (versão 3.12.3). Para o desenvolvimento da solução, foram utilizadas as seguintes bibliotecas:

- **heapq**: implementação de filas de prioridade, essencial para os algoritmos *UCS* e *A**, permitindo acesso eficiente ao elemento de menor custo;
- **math**: operações matemáticas fundamentais, incluindo cálculos de distâncias euclidianas e haversianas;
- **random**: geração de números pseudoaleatórios para seleção probabilística e construção de instâncias de teste;
- **time**: medição precisa do tempo de execução dos algoritmos;
- **copy**: criação de cópias profundas de estruturas de dados, evitando efeitos colaterais indesejados;
- **psutil**: monitoramento do consumo de memória durante a execução dos algoritmos;
- **openpyxl**: exportação dos resultados experimentais para planilhas Excel, facilitando análises posteriores.

4.5.1 Representação do Grafo

A representação computacional do grafo da malha viária foi implementada utilizando duas estruturas complementares armazenadas em arquivos binários: (i) uma lista de adjacência contendo as conexões entre os vértices e as distâncias correspondentes; e (ii) uma lista de coordenadas geográficas (latitude e longitude) de cada vértice. Esta abordagem permite carregamento rápido dos dados e consultas eficientes durante a execução dos algoritmos de busca.

A escolha por arquivos binários justifica-se pela otimização do espaço de armazenamento e pela velocidade de leitura, aspectos cruciais quando se trabalha com grafos de grande porte representando malhas viárias complexas. A estrutura de lista de adjacência garante complexidade espacial $O(V + E)$, adequada para grafos esparsos típicos de redes urbanas.

4.5.2 Padronização dos Experimentos

Para garantir a reprodutibilidade dos experimentos e permitir comparações justas entre diferentes execuções dos algoritmos, foi adotado um conjunto padronizado de sementes aleatórias. As sementes utilizadas correspondem a 18 combinações de dígitos da parte decimal da constante matemática π , ou seja, a cada 6 casas decimais uma nova semente foi gerada: 141592, 653589, 793238, 462643, 383279, 502884, 197169, 399375, 105820, 974944, 592307, 816406, 286208, 998628, 34825, 342117, 67982, 148086, 513282, 306647, 93844, 609550, 582231, 725359, 408128, 481117, 450284, 102701, 938521, 105559.

A escolha da constante π para geração das sementes justifica-se por sua natureza irracional e distribuição uniforme de dígitos, garantindo que as sequências pseudoaleatórias geradas apresentem propriedades estatísticas adequadas para a realização de experimentos científicos. Cada execução dos algoritmos utilizou uma dessas sementes de forma sequencial, permitindo a geração de múltiplas instâncias de teste com características variadas.

4.6 Análise comparativa dos experimentos

A avaliação da solução proposta neste trabalho baseia-se em uma análise comparativa sistemática que visa mensurar a eficiência e a qualidade das rotas geradas pelo algoritmo híbrido desenvolvido. A metodologia de análise adota múltiplas métricas complementares que permitem uma avaliação abrangente do desempenho da solução, considerando tanto aspectos computacionais quanto a qualidade das rotas obtidas.

4.6.1 Métricas de Avaliação

Para a análise comparativa, foram estabelecidas duas métricas principais que refletem os objetivos práticos da otimização de rotas na gestão pública:

- **Tempo de Execução:** Medido em segundos, representa o tempo computacional necessário para que o algoritmo gere uma solução completa. Esta métrica é fundamental para avaliar a viabilidade prática da implementação, especialmente em cenários onde decisões precisam ser tomadas em tempo hábil. Para garantir precisão nas medições, foram utilizadas as funcionalidades da biblioteca *time* do Python, que permite cronometrar com alta resolução o tempo decorrido entre o início e o término da execução de cada algoritmo.
- **Qualidade da Solução:** Quantificada pelo custo total da rota gerada, expresso em unidades de distância (metros). Esta métrica representa a soma das distâncias de todos os trechos que compõem o percurso completo, desde a partida do ponto de origem, passando por todos os ativos a serem visitados, até o retorno ao ponto inicial. Menores valores de custo indicam rotas mais eficientes, que resultam em economia de combustível, redução de tempo de deslocamento e menor desgaste dos veículos.

4.6.2 Estratégia de Comparação

A validação da solução proposta fundamenta-se na comparação com algoritmos de busca clássicos que garantem encontrar o caminho ótimo entre dois pontos em um grafo. Para esta finalidade, foram selecionados os algoritmos *BFS* e *DFS* como referências de comparação. Embora esses algoritmos não sejam especificamente projetados para resolver o *TSP* completo, eles garantem encontrar caminhos válidos entre cada par de ativos, explorando sistematicamente todas as possibilidades de conexão no grafo.

A escolha desses algoritmos como base de comparação justifica-se por suas características complementares. A *DFS* e *BFS*, ao explorar profundamente cada ramo antes de retroceder e em largura oferece uma alternativa com diferentes características de uso de memória e ordem de exploração.

4.6.3 Análise de Viabilidade

A análise de viabilidade da solução proposta será conduzida através da comparação direta entre o desempenho do algoritmo híbrido desenvolvido e os algoritmos de referência. Esta

comparação permitirá verificar se a abordagem heurística consegue gerar soluções competitivas em termos de qualidade, mantendo tempos de execução compatíveis com aplicações práticas.

Espera-se que o algoritmo híbrido, ao combinar estratégias probabilísticas de seleção com técnicas de programação dinâmica, consiga explorar eficientemente o espaço de soluções sem a necessidade de avaliar exaustivamente todas as possibilidades. Esta característica é particularmente relevante para instâncias de maior porte, onde métodos de busca exaustiva tornam-se computacionalmente inviáveis devido ao crescimento exponencial do espaço de busca.

4.6.4 Procedimento Experimental

Os experimentos serão conduzidos sobre múltiplas instâncias de teste, cada uma gerada com uma das sementes aleatórias padronizadas descritas na seção 4.5.2. Para cada instância, serão executados todos os algoritmos de busca selecionados (*BFS*, *DFS*, *UCS* e *A** com diferentes heurísticas), registrando-se os tempos de execução e os custos das rotas obtidas.

Os resultados serão organizados em tabelas e gráficos comparativos que evidenciam as diferenças de desempenho entre as abordagens. Análises estatísticas descritivas, como médias, desvios-padrão e valores extremos, permitirão caracterizar o comportamento geral de cada algoritmo. Adicionalmente, será avaliado o impacto do tamanho da instância (número de ativos a serem visitados) sobre o desempenho de cada abordagem, identificando possíveis limites de escalabilidade.

A partir desta análise comparativa abrangente, será possível não apenas validar a eficácia da solução proposta, mas também fornecer recomendações práticas sobre quais algoritmos de busca são mais adequados para diferentes cenários de aplicação na gestão pública municipal.

4.7 Considerações Finais

Este capítulo apresentou a metodologia adotada para o desenvolvimento da solução de otimização de rotas para visitação de ativos públicos municipais. A abordagem proposta combina técnicas de algoritmos genéticos com programação dinâmica, aplicadas sobre um grafo real da malha viária urbana da cidade de Russas-CE. A revisão sistemática da literatura fundamentou a escolha dos algoritmos de busca (*BFS*, *DFS*, *UCS* e *A**) utilizados para comparação experimental, enquanto a padronização das instâncias de teste, baseada em sementes derivadas

da constante π , garante a reprodutibilidade dos experimentos.

As métricas de avaliação estabelecidas (tempo de execução e qualidade da solução) permitirão analisar não apenas a eficiência computacional dos algoritmos, mas também a viabilidade prática de sua aplicação na gestão pública municipal. A estrutura modular da implementação, com suporte para integração com *APIs* externas de roteamento, confere escalabilidade e adaptabilidade da solução para diferentes contextos urbanos.

O Quadro 2 apresenta o cronograma das atividades realizadas e previstas para a conclusão deste trabalho. As próximas etapas contemplam a análise aprofundada dos resultados experimentais, o desenvolvimento de visualizações gráficas das rotas otimizadas e a construção de uma aplicação visual para demonstração prática da solução, culminando na escrita final do documento.

Quadro 2 – Cronograma de execução - TCC

Atividade	Set/25	Out/25	Nov/25	Dez/25	Jan/26	Fev/26	Mar/26	Abr/26	Mai/26	Jun/26
Revisão da literatura	X	X								
Obtenção das instâncias		X	X							
Implementação dos algoritmos			X	X						
Execução dos experimentos				X	X	X				
Escrita e ajustes TCC1				X	X					
Análise dos resultados						X	X	X		
Desenvolvimento de diagramas e figuras							X	X		
Desenvolvimento de um módulo visual								X	X	
Escrita e ajustes do TCC2								X	X	
Apresentação do TCC										X

Fonte: elaborado pelo autor.

5 RESULTADOS

Nesse capítulo será descrito o ambiente de execução dos experimentos, os resultados obtidos de cada algoritmo, bem como uma análise comparativa entre eles.

5.1 Ambiente Computacional

Os experimentos foram conduzidos em um ambiente computacional com as seguintes especificações técnicas, utilizando o *Windows Subsystem for Linux 2 (WSL2)* sobre o sistema operacional Windows 11:

- **Processador:** AMD Ryzen 5 4600G com Radeon Graphics, arquitetura Zen 2, 6 núcleos físicos com tecnologia *Simultaneous Multithreading* (12 threads), frequência de 3.693 GHz;
- **Memória RAM:** 8 GiB, com 7.7 GiB disponíveis para aplicações;
- **Hierarquia de Cache:** L1 de 384 KiB, L2 de 3 MiB e L3 de 4 MiB;
- **Sistema Operacional:** Ubuntu 24.04.3 LTS (Noble Numbat) executado sobre *kernel* Linux 6.6.87.2-microsoft-standard-WSL2;
- **Compilador:** GCC versão 11.2.0;
- **Armazenamento:** Disco virtual de 1 TiB com sistema de arquivos ext4.

A escolha deste ambiente justifica-se pela disponibilidade de recursos computacionais adequados para a execução dos algoritmos propostos, bem como pela compatibilidade com ferramentas de desenvolvimento em Python. O uso do *WSL2* permite executar nativamente aplicações Linux em ambiente Windows, combinando a flexibilidade do ecossistema Linux com a interface familiar do Windows.

5.2 Execução dos Algoritmos

Os experimentos foram conduzidos de forma automatizada através de um *script* Shell (arquivo .sh) executado no ambiente *WSL2*, permitindo a execução sequencial de todas as configurações de teste planejadas. O *script* principal *Rota.py* foi invocado com diferentes parâmetros para cada combinação de grafo e quantidade de ativos a serem visitados, gerando automaticamente planilhas Excel com os resultados detalhados de cada execução.

5.2.1 Configuração dos Experimentos

A estratégia experimental adotada envolveu a execução dos algoritmos sobre quatro grafos de malha viária com diferentes tamanhos, variando a quantidade de instancias da imagem 9, representados pelos mapas RUSSAS_MAPA_N13, RUSSAS_MAPA_N32, RUSSAS_MAPA_N42 e RUSSAS_MAPA_N81, onde o número após 'N' indica a quantidade de vértices (nós) no grafo. Para cada grafo, foram testadas diferentes quantidades de ativos a serem visitados, variando de 5 a 40 pontos, conforme a dimensão do problema permitisse.

Os cinco algoritmos de busca implementados foram avaliados: *DFS*, *BFS*, *UCS*, *A** com heurística euclidiana e *A** com heurística haversiana. O *script* de automação executou cada algoritmo para todas as combinações de grafos e quantidades de ativos, seguindo o padrão de invocação ilustrado nos comandos comentados abaixo. Cada linha representa uma execução individual do programa principal, especificando a quantidade de entregas, o grafo utilizado, o arquivo de saída e o algoritmo selecionado:

5.2.2 Resultados Preliminares

A Tabela 2 apresenta os resultados preliminares obtidos após a execução dos experimentos. Para cada combinação de grafo e quantidade de ativos, são reportados a distância total da rota gerada (em metros) e o tempo de execução (em segundos) de cada algoritmo. Os valores marcados com #NULL e #ERRO indicam casos em que os algoritmos *DFS* e *BFS* excederam o tempo máximo de execução estabelecido ou esgotaram os recursos computacionais disponíveis, particularmente para o grafo de maior dimensão (N81).

Observa-se que, para todas as instâncias onde houve convergência, todos os algoritmos encontraram rotas com a mesma distância total, indicando que as soluções obtidas são equivalentes em termos de qualidade. No entanto, verifica-se uma diferença significativa nos tempos de execução, especialmente à medida que o tamanho do grafo e a quantidade de ativos aumentam. Os algoritmos *UCS* e *A** (ambas as variantes) demonstraram desempenho temporal substancialmente superior aos algoritmos *DFS* e *BFS*, evidenciando a importância da utilização de estratégias de busca informada para problemas de otimização de rotas em grafos de maior dimensão.

Tabela 2 – Resultados preliminares dos algoritmos de busca em diferentes instâncias do problema

Quantidade de nós	Quantidade de ativo	Busca em Profundidade		Busca em Largura		Busca de Custo Uniforme		A* Euclidiano		A* Herversiano	
		Distância (m)	Tempo (s)	Distância (m)	Tempo (s)	Distância (m)	Tempo (s)	Distância (m)	Tempo (s)	Distância (m)	Tempo (s)
13	5	1053,00	0,01	1053,00	0,012	1053,00	0,011	1053,00	0,011	1053,00	0,011
13	10	1285,00	0,03	1285,00	0,035	1285,00	0,028	1285,00	0,028	1285,00	0,029
32	5	1549,00	4,16	1549,00	11,713	1549,00	0,013	1549,00	0,012	1549,00	0,012
32	10	2066,00	11,10	2066,00	31,542	2066,00	0,030	2066,00	0,028	2066,00	0,029
32	15	2656,00	26,27	2656,00	73,000	2656,00	0,055	2656,00	0,050	2656,00	0,051
32	20	3875,00	46,67	3875,00	131,253	3875,00	0,086	3875,00	0,080	3875,00	0,079
42	5	1914,00	113,71	1914,00	635,459	1914,00	0,013	1914,00	0,012	1914,00	0,012
42	10	3144,00	441,26	3144,00	2.525,536	3144,00	0,032	3144,00	0,030	3144,00	0,030
42	15	4053,00	893,04	4053,00	5.280,027	4053,00	0,059	4053,00	0,053	4053,00	0,054
42	20	5344,00	1319,39	5344,00	7.522,467	5344,00	0,094	5344,00	0,086	5344,00	0,085
42	25	5815,00	1880,65	5815,00	10.520,795	5815,00	0,128	5815,00	0,116	5815,00	0,116
42	30	7159,00	2671,14	7159,00	16.138,764	7159,00	0,175	7159,00	0,157	7159,00	0,158
81	5	#NULL	#ERRO	#NULL	#ERRO	2380,00	0,013	2380,00	0,012	2380,00	0,012
81	10	#NULL	#ERRO	#NULL	#ERRO	3231,00	0,034	3231,00	0,032	3231,00	0,031
81	15	#NULL	#ERRO	#NULL	#ERRO	4881,00	0,064	4881,00	0,053	4881,00	0,054
81	20	#NULL	#ERRO	#NULL	#ERRO	6143,00	0,100	6143,00	0,084	6143,00	0,085
81	25	#NULL	#ERRO	#NULL	#ERRO	8267,00	0,144	8267,00	0,121	8267,00	0,118
81	30	#NULL	#ERRO	#NULL	#ERRO	8837,00	0,198	8837,00	0,168	8837,00	0,158
81	35	#NULL	#ERRO	#NULL	#ERRO	11538,00	0,259	11538,00	0,213	11538,00	0,208
81	40	#NULL	#ERRO	#NULL	#ERRO	12987,00	0,331	12987,00	0,287	12987,00	0,266

Fonte: elaborada pelo autor (2025).

REFERÊNCIAS

- BARTH, M. J. **Otimização multi-nível para projeto de redes híbridas (ópticas e sem fio) para implementação de cidades inteligentes**. Dissertação (Mestrado em Computação Aplicada) — Universidade do Vale do Rio dos Sinos - UNISINOS, São Leopoldo - RS, Brasil, 2016. Orientador: José Vicente Canto dos Santos.
- BOTELHO, P.; BORGES, A.; ALVES, G. Proposta de implantação de um sistema ciberfísico para um smart parking baseado em agentes inteligentes. In: **Anais do XIII Workshop-Escola de Sistemas de Agentes, seus Ambientes e Aplicações**. Porto Alegre, RS, Brasil: SBC, 2019. p. 259–264. ISSN 2326-5434. Disponível em: <<https://sol.sbc.org.br/index.php/wesaac/article/view/33361>>.
- CARVALHO, M. D. S. d. **Smart cities: uso de sensores e dados secundários para cidades inteligentes centradas no cidadão**. Dissertação (Mestrado) — Universidade de Caxias do Sul, Caxias do Sul, Brasil, 2024.
- CASTRO, L.; MANOEL, F.; JESUS, V.; PANTOJA, C.; BORGES, A.; ALVES, G. Integrando sistemas multi-agentes embarcados, simulação urbana e aplicações de iot. In: **Anais do XIV Workshop-Escola de Sistemas de Agentes, seus Ambientes e Aplicações**. Porto Alegre, RS, Brasil: SBC, 2020. p. 165–176. ISSN 2326-5434. Disponível em: <<https://sol.sbc.org.br/index.php/wesaac/article/view/33389>>.
- CHAVES, A. A.; BIAJOLI, F. L.; MINE, O. M.; SOUZA, M. J. F. Metaheurísticas híbridas para resolução do problema do caixeiro viajante com coleta de prêmios. **Produção**, ABEPRO, Brasil, v. 17, n. 2, p. 263–272, 2007.
- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.
- COSTA, I. S. Bacharelado em Engenharia Urbana, **Proposição de elementos para uso de sistemas inteligentes de transportes para potencialização de intervenções estruturantes no sistema viário de cidades de médio porte: uma proposta para o novo anel viário de Bom Despacho-MG**. Ouro Preto - MG, Brasil: [s.n.], 2023.
- DIAS, C.; LOPES, F.; LEITE, J. Smartnode dashboard: um framework front-end baseado em node-red para criação de city dashboards. In: **Anais do II Workshop Brasileiro de Cidades Inteligentes**. Porto Alegre, RS, Brasil: SBC, 2019. ISSN 0000-0000. Disponível em: <<https://sol.sbc.org.br/index.php/wbci/article/view/6744>>.
- GEORGES, M. R. R. Rotas solidárias: um estudo das rotas de coleta de materiais recicláveis numa cooperativa popular de coleta e seleção de recicláveis. **Revista Gestão Industrial**, UTFPR, Ponta Grossa - PR, Brasil, v. 10, n. 1, p. 1–21, 2014.
- KITCHENHAM, B.; BRERETON, O. P.; BUDGEN, D.; TURNER, M.; BAILEY, J.; LINKMAN, S. Systematic literature reviews in software engineering – a systematic literature review. **Information and Software Technology**, v. 51, n. 1, p. 7–15, 2009. ISSN 0950-5849. Special Section - Most Cited Articles in 2002 and Regular Research Papers. Disponível em: <<https://www.sciencedirect.com/science/article/pii/S0950584908001390>>.
- MELONIO, A. C. C. **Smart São Paulo: um estudo da mobilidade urbana sob a ótica de Machine Learning e aspectos espaço-temporais**. Dissertação (Mestrado) — Universidade Presbiteriana Mackenzie, 2021.

OLIVEIRA, V. A. T. **Gestão de operações de serviços de emergência no contexto de cidades inteligentes e sustentáveis: o caso da Polícia Militar do Paraná**. Dissertação (Mestrado) — Universidade Tecnológica Federal do Paraná, 2020.

PAZ, F. A. **Planejamento de Rotas Veiculares e Otimização de Mobilidade Urbana Utilizando Algoritmo Bioinspirado e Paralelo**. Dissertação (Proposta de Dissertação de Mestrado em Ciência da Computação) — Universidade Federal de Sergipe, São Cristóvão – SE, Brasil, 2023. Orientador: Rubens de Souza Matos Júnior; Coorientador: Ricardo José Paiva de Britto Salgueiro.

RUSSELL, S. J.; NORVIG, P. **Inteligência artificial**. 3. ed. Rio de Janeiro: Elsevier, 2013. Tradução de: Artificial intelligence, 3rd ed. ISBN 978-85-352-3701-6.

SANTIAGO, T. E. T. **Cidades inteligentes, gestão urbana e geotecnologias: proposta de city information modeling (CIM) aplicado ao Município de Madre de Deus-BA**. Dissertação (Mestrado) — Universidade Católica do Salvador (UCSal), 2023.

SANTOS, L.; NASCIMENTO, L.; NUNES, S. Reclamando app: um aplicativo para auxiliar na reivindicação de problemáticas urbanas. In: **Anais da XIX Escola Regional de Computação Bahia, Alagoas e Sergipe**. Porto Alegre, RS, Brasil: SBC, 2019. p. 184–189. ISSN 0000-0000. Disponível em: <<https://sol.sbc.org.br/index.php/erbase/article/view/8976>>.

SATI, T. N.; SCARPIN, C. T.; JUNIOR, J. E. P.; LOPES, R. M. Z. Diferentes formulações de programação linear inteira para a roteirização no transporte de funcionários. **Produção**, UFPR, Curitiba - PR, Brasil, v. 17, n. 2, p. 263–272, 2020.

SILVA, A. R. Villela da; OCHI, L. S. Um algoritmo evolutivo para o problema do caixeiro alugador. In: **Proceeding Series of the Brazilian Society of Applied and Computational Mathematics**. Gramado - RS, Brasil: SBMAC, 2017. v. 5, n. 1, p. 460–467.

SILVA, E. C. d.; LIMA, D. R.; SANTOS, L. C. d.; SILVA, F. D. V. d. Uso da ferramenta or-tools na geração de rotas otimizadas em agência dos correios utilizando algoritmos para o problema do caixeiro viajante. In: **Anais de evento acadêmico (IFCE - Campus Aracati)**. Aracati - CE, Brasil: [s.n.], 2022.

SILVA, M. d. O. Bacharelado em Engenharia de Produção, **Desenvolvimento de uma aplicação gráfica para resolução de problemas de roteamento de veículos**. João Pessoa - PB, Brasil: [s.n.], 2024. Orientador: Luciano Carlos Azevedo da Costa.

STORCK, C.; SALES, E.; ZÁRATE, L.; FIGUEIREDO, F. Proposta de um framework baseado em mineração de dados para redes 5g. **Revista Eletrônica de Sistemas de Informação**, v. 16, 2017. ISSN 1677-3071. Disponível em: <<https://www.periodicosibepes.org.br/index.php/reinfo/article/view/2488>>.

WEISS, M. C.; BERNARDES, R. C.; CONSONI, F. L. Cidades inteligentes como nova prática para o gerenciamento dos serviços e infraestruturas urbanos: a experiência da cidade de porto alegre. **Urbe. Revista Brasileira de Gestão Urbana**, SciELO Brasil, v. 7, p. 310–324, 2015.